



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Goa Campus

OPERATING SYSTEMS
CS F372

Custom System Call Implementation

Linux Kernel Version 6.19.10

`sys_memlineage` — Memory Write Tracking Syscall (No. 471)

Team	Team Hyprland
Members	Krutik Rana Kushagra Singh
Kernel Version	Linux 6.19.10
Syscall Number	471 (x86_64)
Environment	VirtualBox + Ubuntu
Working Dir	<code>/usr/src/linux-6.19.10</code>
Date	March 31, 2026

Contents

1	Introduction	2
1.1	Files Modified or Created	2
2	Implementation Files	2
2.1	sys_memlineage.h	2
2.2	sys_memlineage.c	3
3	Environment Setup	5
4	First Kernel Build	7
5	Syscall Integration into the Kernel Tree	8
6	Recompilation and Testing	12
7	Work Division	14
7.1	Krutik Rana	14
7.2	Kushagra Singh	14

1. Introduction

This report documents the step-by-step implementation of a custom Linux kernel system call, `sys_memlineage`, integrated into Linux kernel version 6.19.10. The work was carried out inside a VirtualBox virtual machine running Ubuntu, with `/usr/src` as the working directory.

`sys_memlineage` is assigned syscall number **471** on the x86-64 ABI. It allows user-space processes to register memory regions for write-event tracking, log individual write events into a per-region ring buffer maintained in the kernel, query past events from that ring, and enable or disable tracking per region.

Its prototype, declared in `include/linux/syscalls.h`, is:

```
asmlinkage long sys_memlineage(int cmd, void __user *arg);
```

The `cmd` argument selects one of five operations; `arg` points to a command-specific struct copied from user space via `copy_from_user`.

1.1. Files Modified or Created

File Path	Action	Description
<code>sys_memlineage.c</code>	Created	Core syscall implementation
<code>sys_memlineage.h</code>	Created	Shared header: structs & constants
<code>syscall_64.tbl</code>	Modified	Entry 471 for memlineage
<code>generic-asm/syscalls.h</code>	Modified	<code>#ifndef</code> guard declaration
<code>linux/syscalls.h</code>	Modified	<code>asmlinkage</code> prototype
<code>Makefile</code>	Modified	<code>sys_memlineage.o</code> in <code>obj-y</code>

2. Implementation Files

2.1. `sys_memlineage.h`

Location: `sys_memlineage.h` (≈ 1.0 kB)

This header is shared between the kernel and user-space programs. It uses `#ifdef __KERNEL__` to switch between kernel types and standard C99 types.

Listing 1: `include/linux/sys_memlineage.h` — limits, commands, and structs

```
1  /* Tunable limits */
2  #define ML_MAX_REGIONS 64    /* global cap on tracked regions */
3  #define ML_MAX_EVENTS 32    /* ring-buffer depth per region */
4  #define ML_COMM_LEN 16     /* process name bytes (== TASK_COMM_LEN) */
5
6  /* Command codes -- first argument to the syscall */
7  #define ML_CMD_REGISTER 1    /* register a memory region */
8  #define ML_CMD_LOG_WRITE 2   /* record a write event */
9  #define ML_CMD_QUERY 3      /* read region metadata + event ring */
10 #define ML_CMD_TRACKING_ENABLE 4 /* resume event logging */
11 #define ML_CMD_TRACKING_DISABLE 5 /* pause event logging */
12
13 /* One logged write event */
14 struct ml_event {
```

```

15     u32  pid;           /* thread ID of writer */
16     u32  tgid;          /* process ID of writer */
17     u64  timestamp_ns; /* ktime_get_ns() at time of write */
18     u64  write_offset;  /* byte offset within the region */
19     u64  write_size;    /* number of bytes written */
20     char comm[ML_COMM_LEN]; /* process name (task->comm) */
21 } __attribute__((packed));
22
23 /* Per-region metadata + event ring kept in kernel */
24 struct ml_region {
25     u64 start_addr; /* user-space base address */
26     u64 size;        /* region size in bytes */
27     u32 owner_pid;   /* PID of registering task */
28     u32 owner_uid;   /* UID of registering task */
29     u32 tracking_enabled; /* 1=active, 0=paused */
30     u32 event_count; /* total events ever logged */
31     struct ml_event events[ML_MAX_EVENTS]; /* circular ring buffer */
32 } __attribute__((packed));
33
34 /* Argument structs for each command */
35 struct ml_register_args { u64 start_addr; u64 size; };
36 struct ml_log_write_args {
37     u64 region_addr; u64 write_offset; u64 write_size;
38 };
39 struct ml_query_args {
40     u64 region_addr; u32 event_count; u32 _pad;
41     struct ml_region region_data;
42 };
43 struct ml_tracking_control { u64 region_addr; };
44
45 /* User-space inline wrapper (excluded from kernel builds) */
46 #ifndef __KERNEL__
47 #define MEMLINEAGE_SYSCALL_NR 471
48 static inline long memlineage(int cmd, void *arg) {
49     return syscall(MEMLINEAGE_SYSCALL_NR, cmd, arg);
50 }
51 #endif

```

2.2. `sys_memlineage.c`

Location: `sys_memlineage.c` (≈ 4.5 kB)

This is the core kernel-space implementation. It defines the `ml_region_internal` wrapper struct, a global doubly-linked list `g_tracked_regions` protected by spinlock `g_regions_lock`, and an atomic region counter. Five command handlers are dispatched inside `SYSCALL_DEFINE2`:

- **ML_CMD_REGISTER** — `kmallocs` a new region, checks address collision (`-EEXIST`) and the 64-region cap (`-ENOSPC`), appends to the linked list under the spinlock.
- **ML_CMD_LOG_WRITE** — locates the region, checks ownership and tracking state (`-EPERM` if disabled), validates bounds (`-ERANGE`), writes a timestamped `ml_event` at index `event_count % ML_MAX_EVENTS`.
- **ML_CMD_QUERY** — copies the `ml_region` snapshot to user space. Non-root callers see only their own events (filtered by `tgid`); `CAP_SYS_ADMIN` holders see all.

- **ML_CMD_TRACKING_ENABLE/DISABLE** — toggles tracking_enabled on the target region under the spinlock.

Listing 2: kernel/sys_memlineage.c — structures and entry point (abridged)

```

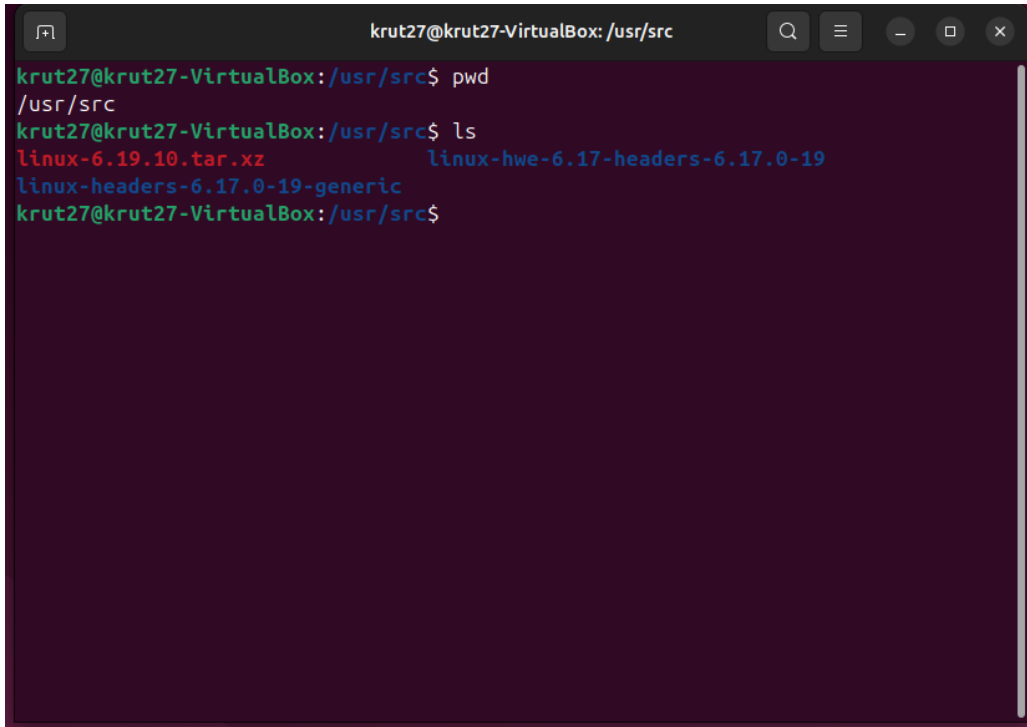
1 #include <linux/kernel.h>
2 #include <linux/syscalls.h>
3 #include <linux/uaccess.h>
4 #include <linux/spinlock.h>
5 #include <linux/list.h>
6 #include <linux/slab.h>
7 #include <linux/ktime.h>
8 #include <linux/cred.h>
9 #include <linux/capability.h>
10 #include <linux/sys_memlineage.h>
11
12 /* Internal per-region wrapper stored in kernel memory */
13 struct ml_region_internal {
14     struct list_head node; /* links into g_tracked_regions */
15     struct ml_region data; /* region metadata + event ring */
16     kuid_t owner_kuid; /* kernel UID of registering process */
17 };
18
19 static LIST_HEAD(g_tracked_regions); /* global region list */
20 static DEFINE_SPINLOCK(g_regions_lock); /* protects the list */
21 static atomic_t g_region_count = ATOMIC_INIT(0);
22
23 /* Finds region whose range contains addr; O(N), N <= 64 */
24 static struct ml_region_internal *ml_find_region(u64 addr) { ... }
25
26 /* Returns true iff the caller is the region owner */
27 static bool ml_is_owner(const struct ml_region_internal *r,
28                        const struct cred *cred) { ... }
29
30 /* Syscall entry point */
31 SYSCALL_DEFINE2(memlineage, int, cmd, void __user *, arg)
32 {
33     const struct cred *cred = get_current_cred();
34     long ret = 0;
35     if (!arg) { put_cred(cred); return -EFAULT; }
36     switch (cmd) {
37     case ML_CMD_REGISTER:
38         ret = ml_register_region(&kargs, cred); break;
39     case ML_CMD_LOG_WRITE:
40         ret = ml_log_write_event(&kargs, cred); break;
41     case ML_CMD_QUERY:
42         ret = ml_query_region(&kargs, cred); break;
43     case ML_CMD_TRACKING_ENABLE:
44     case ML_CMD_TRACKING_DISABLE:
45         ret = ml_set_tracking_state(...); break;
46     default:
47         ret = -EINVAL;
48     }
49     put_cred(cred);
50     return ret;
51 }

```

3. Environment Setup

Step 1 — Download Kernel 6.19.10

The kernel source archive `linux-6.19.10.tar.xz` was downloaded into `/usr/src`, the root working directory for the project.

A terminal window titled 'krut27@krut27-VirtualBox: /usr/src' showing the output of 'pwd' and 'ls' commands. The 'ls' command lists several files in the directory, including 'linux-6.19.10.tar.xz' and 'linux-hwe-6.17-headers-6.17.0-19'.

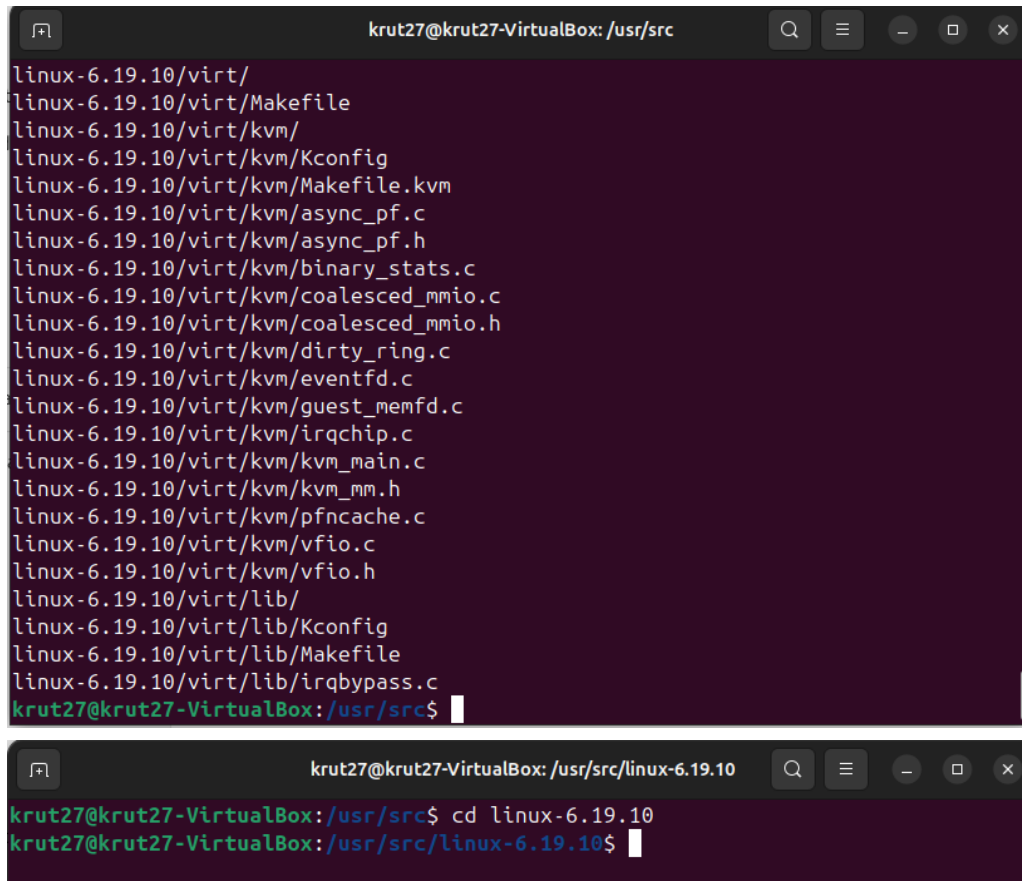
```
krut27@krut27-VirtualBox:/usr/src$ pwd
/usr/src
krut27@krut27-VirtualBox:/usr/src$ ls
linux-6.19.10.tar.xz          linux-hwe-6.17-headers-6.17.0-19
linux-headers-6.17.0-19-generic
krut27@krut27-VirtualBox:/usr/src$
```

Figure 1: `/usr/src` showing the downloaded kernel archive

Step 2 — Extract the Kernel Source

```
tar -xvf linux-6.19.10.tar.xz
```

This produces `/usr/src/linux-6.19.10/`.



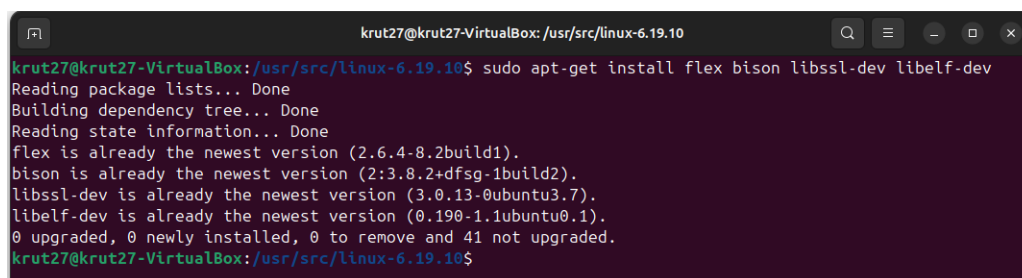
```
krut27@krut27-VirtualBox: /usr/src
linux-6.19.10/virt/
linux-6.19.10/virt/Makefile
linux-6.19.10/virt/kvm/
linux-6.19.10/virt/kvm/Kconfig
linux-6.19.10/virt/kvm/Makefile.kvm
linux-6.19.10/virt/kvm/async_pf.c
linux-6.19.10/virt/kvm/async_pf.h
linux-6.19.10/virt/kvm/binary_stats.c
linux-6.19.10/virt/kvm/coalesced_mmio.c
linux-6.19.10/virt/kvm/coalesced_mmio.h
linux-6.19.10/virt/kvm/dirty_ring.c
linux-6.19.10/virt/kvm/eventfd.c
linux-6.19.10/virt/kvm/guest_memfd.c
linux-6.19.10/virt/kvm/irqchip.c
linux-6.19.10/virt/kvm/kvm_main.c
linux-6.19.10/virt/kvm/kvm_mm.h
linux-6.19.10/virt/kvm/pfncache.c
linux-6.19.10/virt/kvm/vfio.c
linux-6.19.10/virt/kvm/vfio.h
linux-6.19.10/virt/lib/
linux-6.19.10/virt/lib/Kconfig
linux-6.19.10/virt/lib/Makefile
linux-6.19.10/virt/lib/irqbypass.c
krut27@krut27-VirtualBox: /usr/src$

krut27@krut27-VirtualBox: /usr/src/linux-6.19.10
krut27@krut27-VirtualBox: /usr/src$ cd linux-6.19.10
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$
```

Figure 2: Kernel source extraction

Step 3 — Install Build Dependencies

```
sudo apt-get install flex bison libssl-dev libelf-dev
```



```
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$ sudo apt-get install flex bison libssl-dev libelf-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
flex is already the newest version (2.6.4-8.2build1).
bison is already the newest version (2:3.8.2+dfsg-1build2).
libssl-dev is already the newest version (3.0.13-0ubuntu3.7).
libelf-dev is already the newest version (0.190-1.1ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$
```

Figure 3: Build dependencies — all packages already at latest versions

Step 4 — Configure the Kernel

```
sudo make menuconfig      # interactive TUI (select one)
sudo make defconfig       # architecture defaults (used here)
```

make defconfig was used, producing .config from x86_64_defconfig.

```

krut27@krut27-VirtualBox: /usr/src/linux-6.19.10
.config - Linux/x86 6.19.10 Kernel Configuration

Linux/x86 6.19.10 Kernel Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
excluded <M> module < > module capable

[*] General setup --->
  [*] 64-bit kernel
    Processor type and features --->
  [*] Mitigations for CPU vulnerabilities --->
    Power management and ACPI options --->
    Bus options (PCI etc.) --->
    Binary Emulations --->
  [*] Virtualization --->
    General architecture-dependent options --->
  [*] Enable loadable module support --->
  ↓(+)

<select> < Exit > < Help > < Save > < Load >

krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$ sudo make defconfig
HOSTCC  scripts/kconfig/conf.o
HOSTLD  scripts/kconfig/conf
*** Default configuration is based on 'x86_64_defconfig'
#
# configuration written to .config
#
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$

```

Figure 4: Kernel configuration

4. First Kernel Build

Step 5 — Compile the Kernel

```
sudo make -j$(nproc)
```

```

krut27@krut27-VirtualBox: /usr/src/linux-6.19.10
HOSTCC arch/x86/boot/compressed/mkpiggy
CC arch/x86/boot/compressed/cpuflags.o
CC arch/x86/boot/cpu.o
CC arch/x86/boot/compressed/early_serial_console.o
CC arch/x86/boot/compressed/kaslr.o
CC arch/x86/boot/compressed/ident_map_64.o
CC arch/x86/boot/compressed/idt_64.o
AS arch/x86/boot/compressed/idt_handlers_64.o
CC arch/x86/boot/compressed/pgtable_64.o
CC arch/x86/boot/compressed/acpi.o
CC arch/x86/boot/compressed/efi.o
CC arch/x86/boot/compressed/misc.o
GZIP arch/x86/boot/compressed/vmlinux.bin.gz
MKPIGGY arch/x86/boot/compressed/piggy.S
AS arch/x86/boot/compressed/piggy.o
LD arch/x86/boot/compressed/vmlinux
ZOFFSET arch/x86/boot/zoffset.h
OBJCOPY arch/x86/boot/vmlinux.bin
AS arch/x86/boot/header.o
LD arch/x86/boot/setup.elf
OBJCOPY arch/x86/boot/setup.bin
BUILD arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#1)
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$

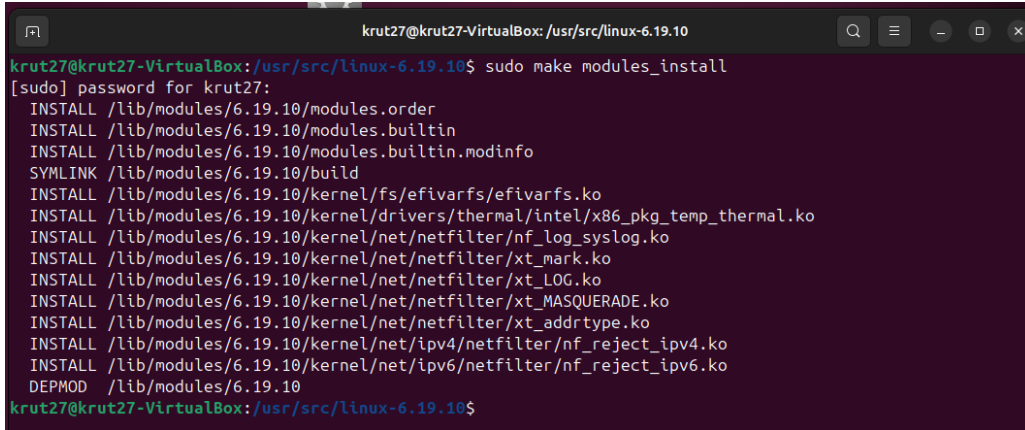
```

Figure 5: Compilation complete — arch/x86/boot/bzImage ready (Build #1)

Step 6 — Install Modules

```
sudo make modules_install
```

Modules are installed to `/lib/modules/6.19.10/`.



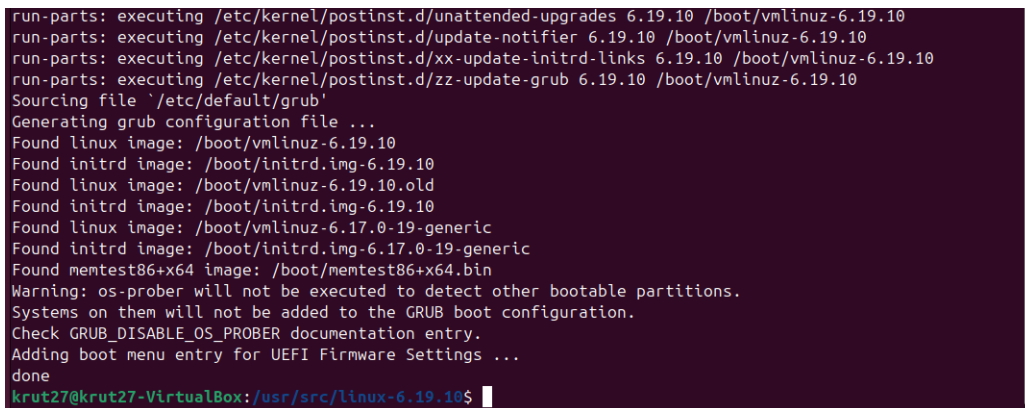
```
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$ sudo make modules_install
[sudo] password for krut27:
INSTALL /lib/modules/6.19.10/modules.order
INSTALL /lib/modules/6.19.10/modules.builtin
INSTALL /lib/modules/6.19.10/modules.builtin.modinfo
SYMLINK /lib/modules/6.19.10/build
INSTALL /lib/modules/6.19.10/kernel/fs/efivarfs/efivarfs.ko
INSTALL /lib/modules/6.19.10/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/nf_log_syslog.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_mark.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_LOG.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_MASQUERADE.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_addrtype.ko
INSTALL /lib/modules/6.19.10/kernel/net/ipv4/netfilter/nf_reject_ipv4.ko
INSTALL /lib/modules/6.19.10/kernel/net/ipv6/netfilter/nf_reject_ipv6.ko
DEPMOD /lib/modules/6.19.10
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$
```

Figure 6: Output of `make modules_install`

Step 7 — Install the Kernel

```
sudo make install
```

The image is placed at `/boot/vmlinuz-6.19.10` and GRUB is regenerated.



```
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.19.10 /boot/vmlinuz-6.19.10
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.19.10 /boot/vmlinuz-6.19.10
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.19.10 /boot/vmlinuz-6.19.10
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.19.10 /boot/vmlinuz-6.19.10
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.19.10
Found initrd image: /boot/initrd.img-6.19.10
Found linux image: /boot/vmlinuz-6.19.10.old
Found initrd image: /boot/initrd.img-6.19.10
Found linux image: /boot/vmlinuz-6.17.0-19-generic
Found initrd image: /boot/initrd.img-6.17.0-19-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$
```

Figure 7: GRUB configuration regenerated after `make install`

5. Syscall Integration into the Kernel Tree

Step 8 — Update Makefile

File: Makefile

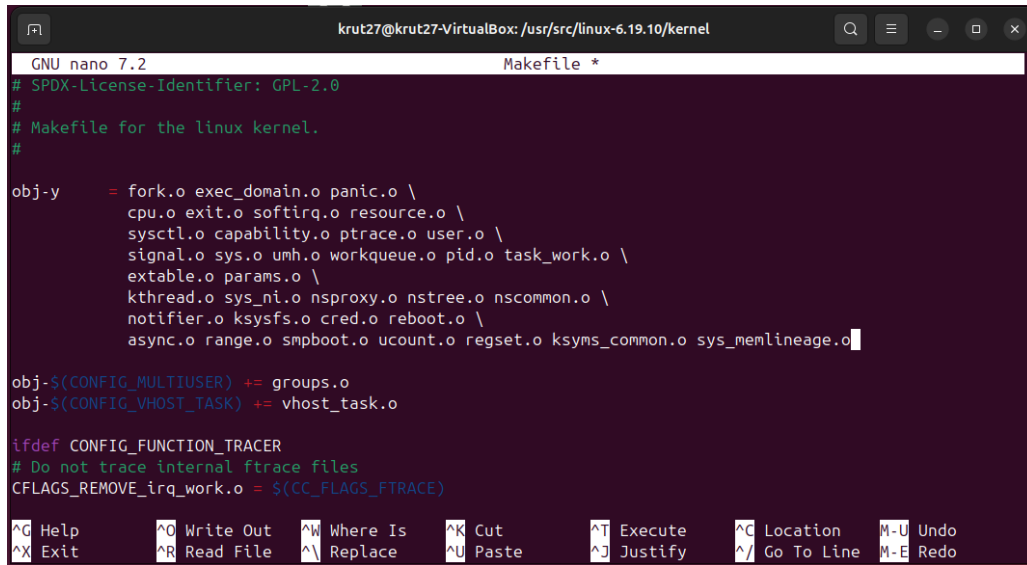
Change: Appended `sys_memlineage.o` at the end of the `obj-y` list so the file is compiled unconditionally into the kernel image.

```
1 obj-y = fork.o exec_domain.o panic.o \
2         cpu.o exit.o softirq.o resource.o \
3         ...
```

```

4   async.o range.o smpboot.o ucount.o regset.o \
5   ksyms_common.o sys_memlineage.o # <-- added

```

Figure 8: kernel/Makefile — *sys_memlineage.o* appended to *obj-y*

Step 9 — Update the x86-64 Syscall Table

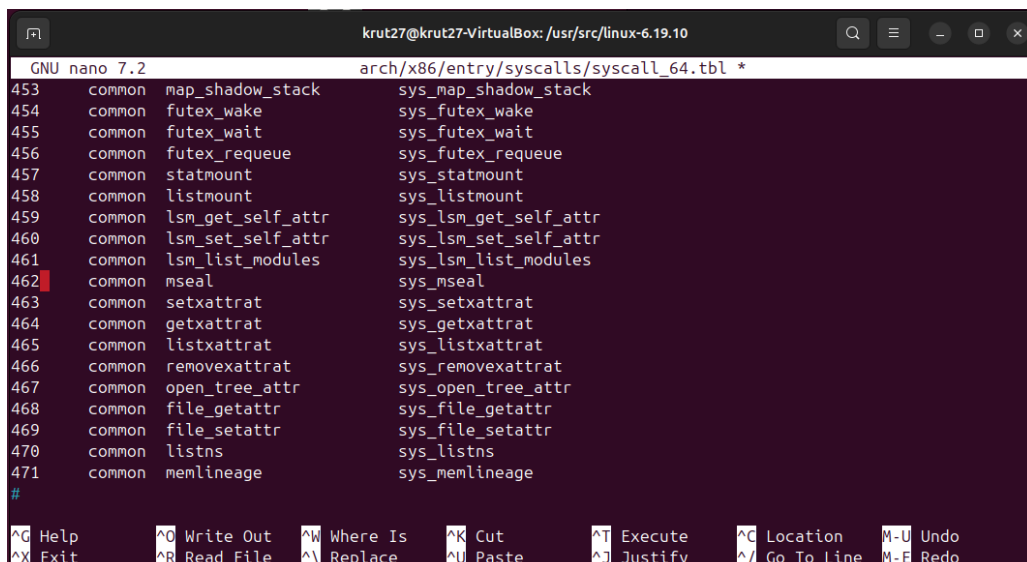
File: *syscall_64.tbl*

Change: Added entry 471 after *listns* (470), the last existing call. The build system uses this table to auto-generate the `__x64_sys_memlineage` dispatch stub.

```

1 471 common memlineage sys_memlineage

```

Figure 9: *syscall_64.tbl* — entry 471 added at end of common range

Step 10 — Update generic-asm/syscalls.h

File: *generic-asm/syscalls.h*

Change: Appended a guarded declaration so architectures that override the calling convention are unaffected.

```
1 #ifndef sys_memlineage
2 asmlinkage long sys_memlineage(int cmd, void __user *arg);
3 #endif
```

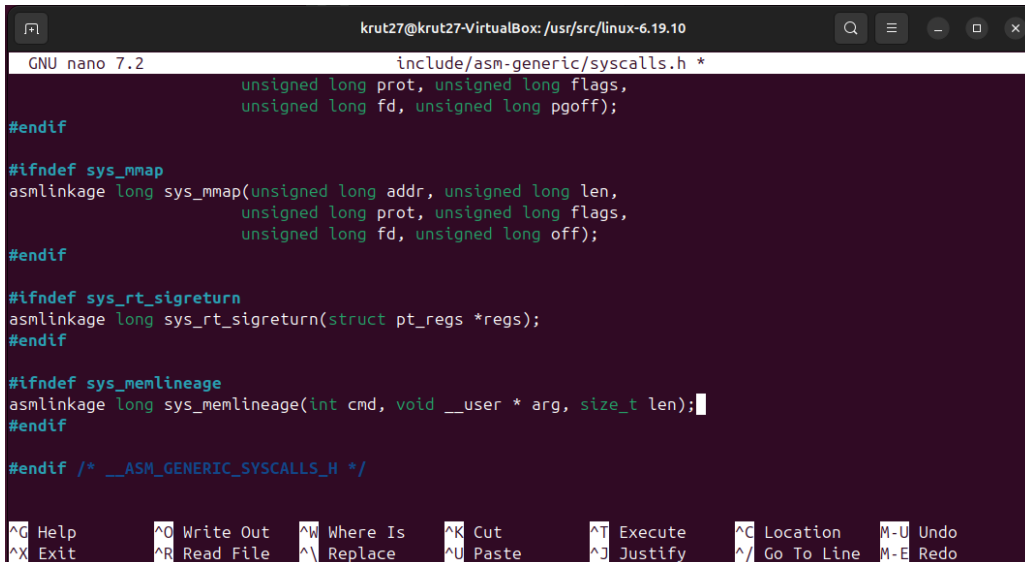


Figure 10: generic-asm/syscalls.h — guarded declaration added

Step 11 — Update linux/syscalls.h

File: linux/syscalls.h

Change: Added the asmlinkage prototype so every kernel compilation unit that includes this header can resolve the declaration.

```
1 asmlinkage long sys_memlineage(int cmd, void __user *arg);
```

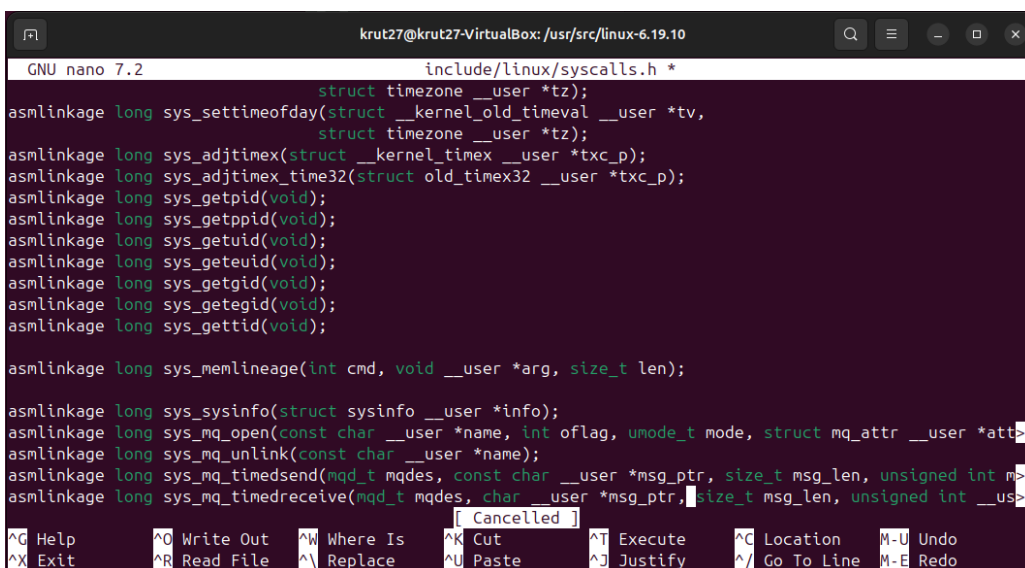


Figure 11: linux/syscalls.h — prototype added

Step 12 — Place `sys_memlineage.c` in project root**File: `sys_memlineage.c`**

The complete implementation file (described in Section 2.1) is stored in the project root alongside the other source files.



Figure 12: File manager view of project root — `sys_memlineage.c` highlighted

Step 13 — Place `sys_memlineage.h` in `linux/`**File: `linux/sys_memlineage.h`**

The shared header (described in Section 2.2) is placed in the `linux/` folder so it is accessible to both the kernel and user-space test programs.

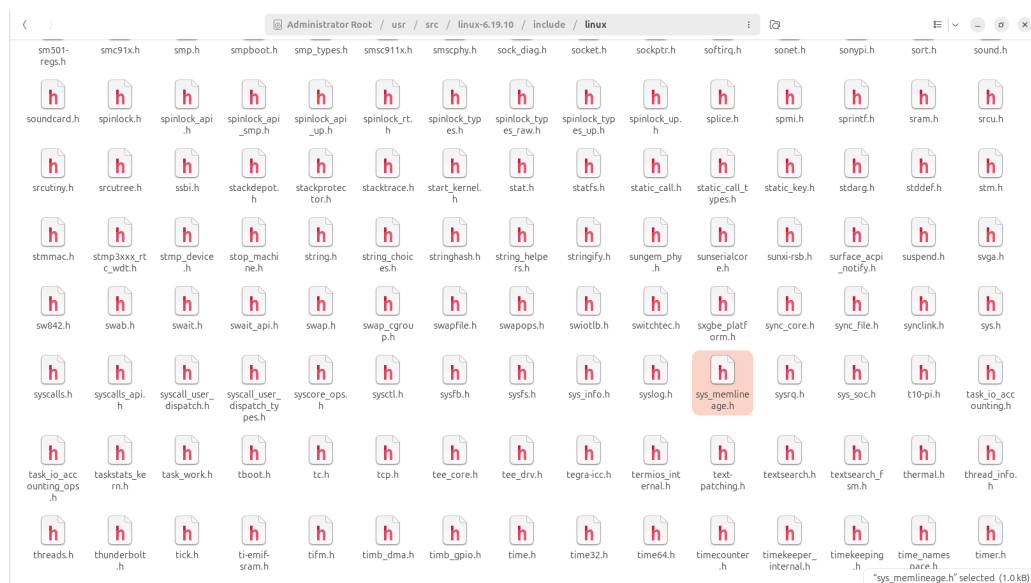


Figure 13: File manager view of `linux/` — `sys_memlineage.h` highlighted

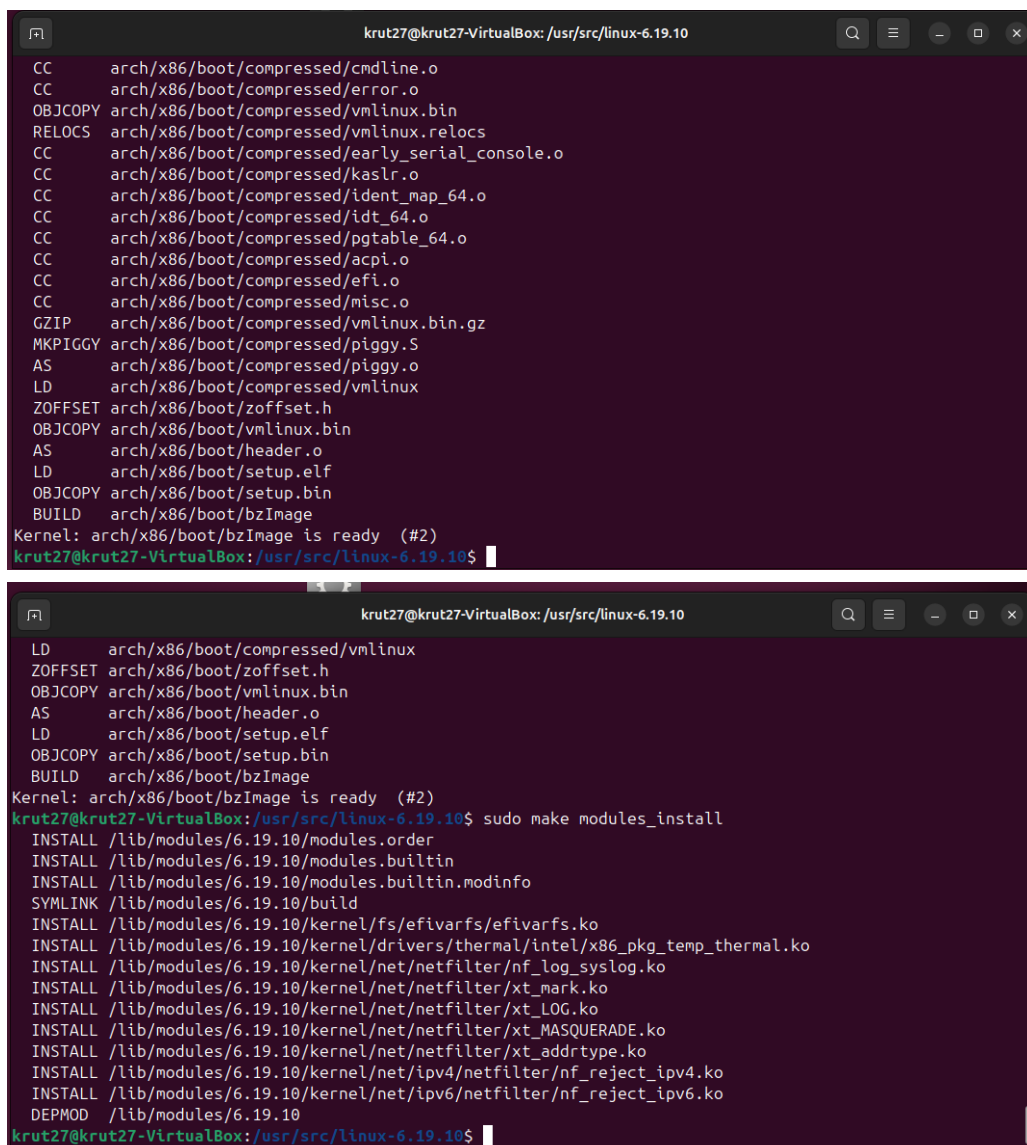
6. Recompilation and Testing

Step 14 — Recompile the Kernel with the New Syscall

After all source changes are in place, the kernel is rebuilt (Build #2):

```
1 sudo make -j$(nproc)
2 sudo make modules_install
3 sudo make install
```

GRUB is updated again; on next boot `/boot/vmlinuz-6.19.10` contains the compiled `sys_memlineage` handler.



```
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10
CC      arch/x86/boot/compressed/cmdline.o
CC      arch/x86/boot/compressed/error.o
OBJCOPY arch/x86/boot/compressed/vmlinux.bin
RELOCS   arch/x86/boot/compressed/vmlinux.relocs
CC      arch/x86/boot/compressed/early_serial_console.o
CC      arch/x86/boot/compressed/kaslr.o
CC      arch/x86/boot/compressed/ident_map_64.o
CC      arch/x86/boot/compressed/idt_64.o
CC      arch/x86/boot/compressed/pgtable_64.o
CC      arch/x86/boot/compressed/acpi.o
CC      arch/x86/boot/compressed/efi.o
CC      arch/x86/boot/compressed/misc.o
GZIP     arch/x86/boot/compressed/vmlinux.bin.gz
MKPIGGY  arch/x86/boot/compressed/piggy.S
AS       arch/x86/boot/compressed/piggy.o
LD       arch/x86/boot/compressed/vmlinux
ZOFFSET  arch/x86/boot/zoffset.h
OBJCOPY  arch/x86/boot/vmlinux.bin
AS       arch/x86/boot/header.o
LD       arch/x86/boot/setup.elf
OBJCOPY  arch/x86/boot/setup.bin
BUILD    arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#2)
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$

krut27@krut27-VirtualBox: /usr/src/linux-6.19.10
LD      arch/x86/boot/compressed/vmlinux
ZOFFSET arch/x86/boot/zoffset.h
OBJCOPY arch/x86/boot/vmlinux.bin
AS      arch/x86/boot/header.o
LD      arch/x86/boot/setup.elf
OBJCOPY arch/x86/boot/setup.bin
BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#2)
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$ sudo make modules_install
INSTALL /lib/modules/6.19.10/modules.order
INSTALL /lib/modules/6.19.10/modules.builtin
INSTALL /lib/modules/6.19.10/modules.builtin.modinfo
SYMLINK /lib/modules/6.19.10/build
INSTALL /lib/modules/6.19.10/kernel/fs/efivarfs/efivarfs.ko
INSTALL /lib/modules/6.19.10/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/nf_log_syslog.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_mark.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_LOG.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_MASQUERADE.ko
INSTALL /lib/modules/6.19.10/kernel/net/netfilter/xt_addrtype.ko
INSTALL /lib/modules/6.19.10/kernel/net/ipv4/netfilter/nf_reject_ipv4.ko
INSTALL /lib/modules/6.19.10/kernel/net/ipv6/netfilter/nf_reject_ipv6.ko
DEPMOD  /lib/modules/6.19.10
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$
```

Figure 14: Second kernel build and installation

```

run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.19.10 /boot/vmlinuz-6.19.10
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.19.10 /boot/vmlinuz-6.19.10
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.19.10 /boot/vmlinuz-6.19.10
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.19.10 /boot/vmlinuz-6.19.10
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.19.10
Found initrd image: /boot/initrd.img-6.19.10
Found linux image: /boot/vmlinuz-6.19.10.old
Found initrd image: /boot/initrd.img-6.19.10
Found linux image: /boot/vmlinuz-6.17.0-19-generic
Found initrd image: /boot/initrd.img-6.17.0-19-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
krut27@krut27-VirtualBox: /usr/src/linux-6.19.10$

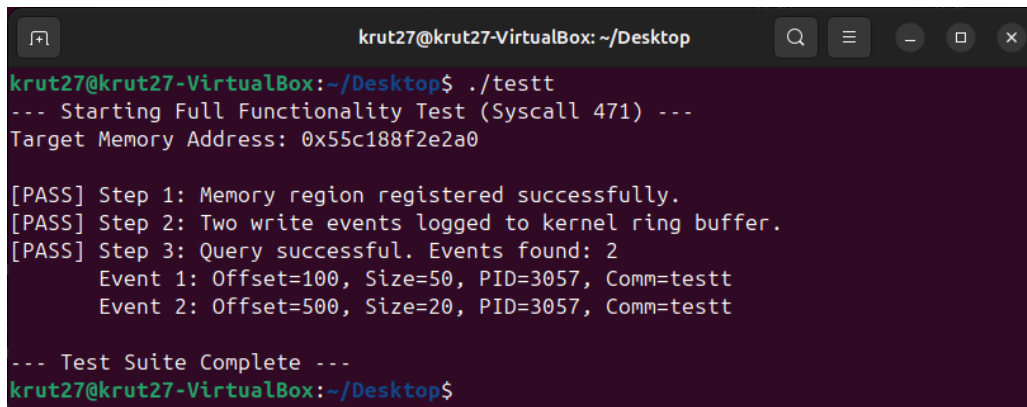
```

Figure 15: GRUB configuration regenerated after second `make install` — Build #2

Step 15 — Test the Syscall

The test suite `syscall_test.c` covers four scenarios:

1. **Registration and Collisions** — registering the same address twice returns `EEXIST`.
2. **Bounds Checking** — `write_offset + write_size` exceeding the region boundary returns `ERANGE`.
3. **Tracking Controls** — `ML_CMD_LOG_WRITE` returns `EPERM` while disabled; returns 0 after re-enabling.
4. **Ring Buffer Wrap-Around** — 40 events into a 32-slot ring confirm circular overwrite via `ML_CMD_QUERY`.



```

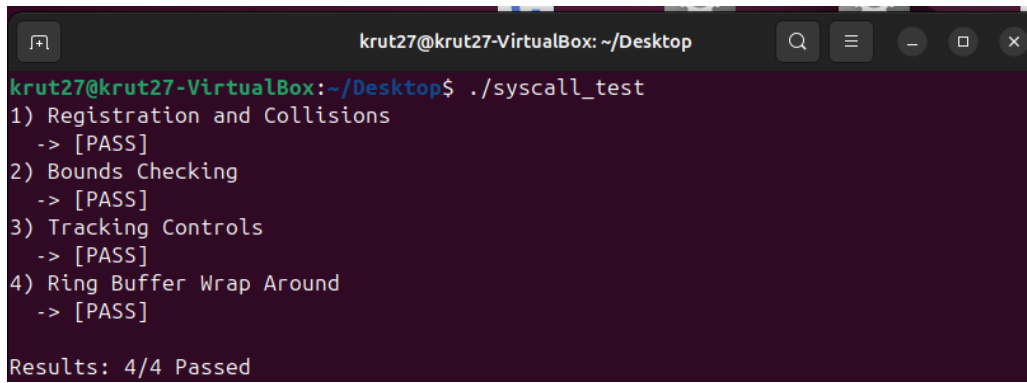
krut27@krut27-VirtualBox: ~/Desktop
krut27@krut27-VirtualBox:~/Desktop$ ./testtt
--- Starting Full Functionality Test (Syscall 471) ---
Target Memory Address: 0x55c188f2e2a0

[PASS] Step 1: Memory region registered successfully.
[PASS] Step 2: Two write events logged to kernel ring buffer.
[PASS] Step 3: Query successful. Events found: 2
        Event 1: Offset=100, Size=50, PID=3057, Comm=testtt
        Event 2: Offset=500, Size=20, PID=3057, Comm=testtt

--- Test Suite Complete ---
krut27@krut27-VirtualBox:~/Desktop$

```

Figure 16: `./testtt` — Full Functionality Test: 3/3 steps PASS



```
krut27@krut27-VirtualBox: ~/Desktop
krut27@krut27-VirtualBox:~/Desktop$ ./syscall_test
1) Registration and Collisions
  -> [PASS]
2) Bounds Checking
  -> [PASS]
3) Tracking Controls
  -> [PASS]
4) Ring Buffer Wrap Around
  -> [PASS]
Results: 4/4 Passed
```

Figure 17: `./syscall_test` — 4/4 tests PASS

7. Work Division

The project was carried out collaboratively by two members, each contributing an equally substantive share of the overall work.

7.1. Krutik Rana

Kernel Environment and Build Pipeline. Krutik set up the full kernel build environment inside the VirtualBox virtual machine. This involved downloading the Linux 6.19.10 source archive, extracting it under `/usr/src`, installing build dependencies (`flex`, `bison`, `libssl-dev`, `libelf-dev`), and running `make defconfig`. He performed both kernel compilation cycles (`make`, `make modules_install`, `make install`) and verified the GRUB configuration after each install.

Syscall Registration and Core Implementation. Krutik edited the x86-64 syscall table (`syscall_64.tbl`) to assign number 471, and updated `Makefile` to include `sys_memlineage.o` in the unconditional build list. He authored the majority of `sys_memlineage.c`: the entry point `SYSCALL_DEFINE2`, the global linked-list and spinlock infrastructure (`g_tracked_regions`, `g_regions.lock`), and the `ML_CMD_REGISTER` and `ML_CMD_LOG_WRITE` handlers including all bounds and ownership checks.

7.2. Kushagra Singh

Interface Design and Header Files. Kushagra designed the user-kernel interface and authored `sys_memlineage.h`: all packed structs, the tunable constants (`ML_MAX_REGIONS=64`, `ML_MAX_EVENTS=32`, `ML_COMM_LEN=16`), command macros, the error-code contract, and the inline `memlineage()` wrapper for user-space programs. He also updated `generic-asm/syscalls.h` with the guarded declaration, and `linux/syscalls.h` with the `asmlinkage` prototype.

Query, Tracking Controls, and Test Suite. Kushagra implemented the `ML_CMD_QUERY`, `ML_CMD_TRACKING_ENABLE`, and `ML_CMD_TRACKING_DISABLE` handlers in `sys_memlineage.c`, including the privilege-aware event filtering logic (UID/TGID check versus `CAP_SYS_ADMIN`). He wrote `syscall_test.c`, the four-test verification suite covering registration collisions, bounds checking, tracking state transitions, and ring-buffer wrap-around behaviour. He compiled and ran both test programs against the final kernel build, confirming all tests pass.